

Software Dependability

Dynamic Techniques to Detect and Exploit Software Vulnerabilities

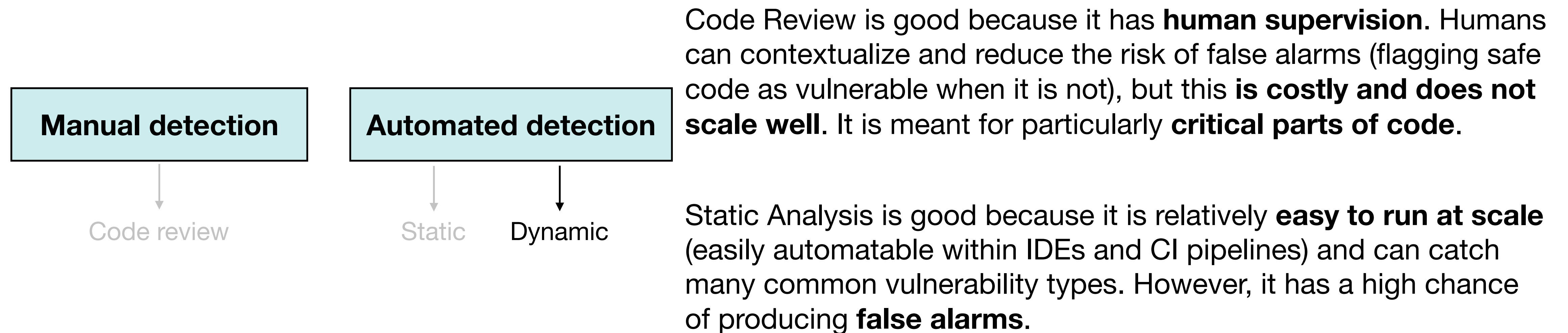
Vulnerabilities and Exploits

A vulnerability is **an instance of fault** in software specification, development, or configuration such that its execution can violate the (implicit or explicit) security policy defined within an organization.

An exploit is a **piece of software** that takes advantage of a bug or vulnerability to cause unintended behavior on computer software or hardware.

Detecting Vulnerabilities

Last time we saw the main methods for detecting software vulnerabilities.



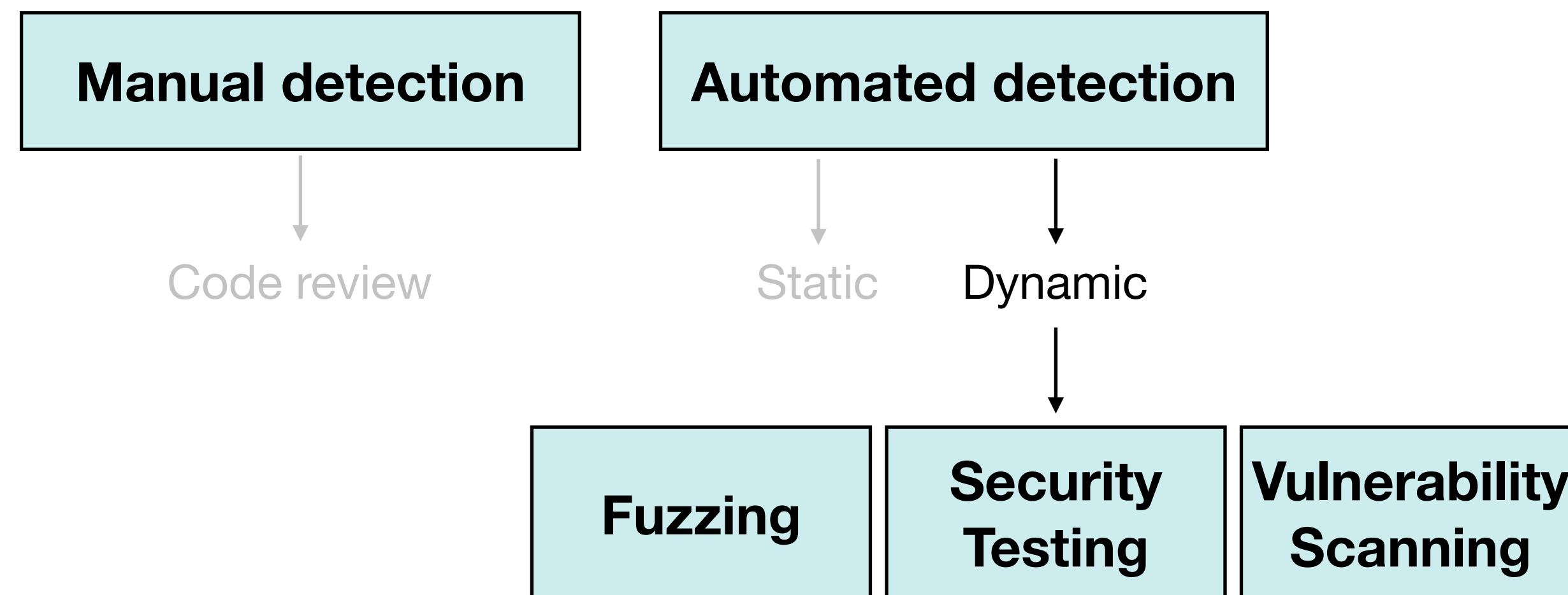
Code Review and Static Analysis **do not execute the code under inspection**.

However, a vulnerability is an **executable fault** that attackers can exploit. Thus, executing some code from the system under test would be logical to discover additional vulnerabilities that other methods miss.

Detecting Vulnerabilities

In today's lecture, we see the main **dynamic testing techniques**, whose goal is to send real inputs to the system—or its components—and observe its responses to determine whether there are security issues.

We can identify subtypes of dynamic testing techniques, with some overlap between them.



Fuzzing in short:

Seeding random input to the system to uncover unconventional behaviors.

Security Testing in short:

Define and execute test cases to (1) ensure specific security properties always hold or (2) no security-threatening conditions happen.

Vulnerability Scanning in short:

Run ready-to-use tools that try known attack patterns to ensure the system has no vulnerabilities of certain types.

Fuzzing

In 1988, Barton Miller, a professor at the University of Wisconsin-Madison, assigned his student an exercise to write a small program that could “fuzz” various UNIX utilities (CLI tools) with random inputs to see which crashed or exhibited other weird behaviors.

A crash happens when a program stops its execution abruptly and returns an error.

When and why does a crash happen? The most common scenarios:

- If the input is fed directly into a buffer and is too large, it may access reserved memory areas, likely causing the infamous Segmentation Fault.
- If the input dictates the number of resources to allocate—e.g., buffer size, number of files to create, and network requests—the system can become overwhelmed and slow down until it runs out of available resources and crashes.
- If the input causes an unexpected exception, it will eventually reach the runtime (e.g., the JVM in Java), which by default kills the program.

Miller assumed that with sufficiently random inputs, any program would eventually crash... and he was right!

Fuzzing is now a trendy dynamic analysis technique for security.

Nevertheless, how are crashes related to security?

Fuzzing and its Relationship with Security

The exercise did not concern security but robustness, i.e., the capability of the utilities to withstand **unusual inputs**.

What is the relationship between robustness and security?

Suppose an application server runs on a computer accessible from the public Internet. What happens if someone submits user input that causes the server to crash?

Our server would stop, and we would stop making a profit for minutes/hours.

Imagine how many millions of euros Amazon would lose if someone managed to take down its servers for hours.

A server interruption is known as a Denial-of-Service (DoS) attack.

Software security is not only a matter of “data theft” or “spying” but also of “making software work under all conditions”.

When we talk about security, we refer to the CIA triad:

- Confidentiality: avoid disclosing sensitive data to unauthorized parties.
- Integrity: keeping data trustworthy and consistent from accidental or intentional alterations.
- Availability: ensuring programs and data are always available when authorized parties request.



Fuzzing and its Relationship with Security

Furthermore, crashes can also be symptoms of other kinds of vulnerabilities.

Let us recall the first crash scenario described earlier:

If the input is fed directly into a buffer and is too large, it may access reserved memory areas, likely causing the infamous *Segmentation Fault*.

This crash occurred due to a Buffer Overflow vulnerability!

In the worst-case scenario, buffer overflows can be exploited to **inject arbitrary code**. Buffer overflows are likely to open the door to developers' worst nightmare: Remote Code Execution (RCE), i.e., when attackers can execute arbitrary code on a remote machine by sending specific inputs.

However, a DoS is not only caused by crashes.

Miller did not only expect crashes from the tested utilities but also other odd behaviors, such as infinite loops.

Such a scenario slows the host machine and requires human intervention to kill the program.

Imagine a server running in the cloud in an infinite loop for hours... the bill at the end of the month will be HEAVY!

So, are you convinced that robustness is a critical aspect of software security?

How does a Fuzzer Work?

A fuzzer essentially does three things:

1. Creates a set of random inputs according to a given strategy.
2. Sends each input to its target (program, function, method, whatever).
3. Observes the response of the target and records everything considered interesting (e.g., crashes, infinite loops).

The real challenge is the first point: how does a fuzzer generate the random input?

How does a Fuzzer Generate the Random Input?

Pure random. It was the first form of fuzzing. It solely relies on pseudo-random number generators, nothing more. Very easy to implement, but it is “too random.”

Suppose we want to fuzz a method that validates XML files, which is well implemented but throws an unhandled exception when an XML file is valid.

Randomly generating files, we will unlikely create a valid XML file: our fuzzer has a low chance of causing crashes.

Mutational Fuzzing. The user provides the first input (i.e., *the seed*): a valid input for the target. Then, the fuzzer applies random mutations to this input and repeatedly sends it to the system. This action increases our chances of receiving valid input while remaining sufficiently random to cause some unconventional behaviors.

The seed can be made of N inputs, not just one.

Greybox Fuzzing. White-box information (e.g., code coverage) is used to improve Mutational Fuzzing.

If some inputs in the seed or certain mutation operators achieve higher coverage than others, they are preferred.

Fuzzing is an active research area, and many techniques are still emerging.

How to Start a Fuzzing Campaign?



What do we fuzz? First, we need to decide our target: do we want to test an entire program or just individual functions/methods? Commonly, fuzzers target the whole system, so the inputs are sent through everything users can use (e.g., command-line arguments and HTML forms).



When do we fuzz? Whenever we have a working piece of software, we do not necessarily wait until later stages of development. Integrating it within CI pipelines (i.e., *Continuous Fuzzing*) is recommended.



For how long do we fuzz? We set a *budget*, e.g., stopping after observing X crashes or after Y minutes. It all depends on the desired amount of robustness we need.



Where do we fuzz? Given its unpredictable nature, it should be run in an *isolated and controlled environment*, e.g., a virtual machine or a container. Imagine what happens if a random input causes the removal of your files on the hard disk!



What do we use to fuzz? We could implement a custom fuzzer. Alternatively, we can rely on well-established solutions, such as the well-known *American Fuzzy Lop ++ (AFL++)*, a greybox fuzzer targeting binary (compiled) programs. Providing the target source code to enable additional analyses is also possible.

American Fuzzy Lop plus plus (AFL++)

american fuzzy lop ++2.65d (libpng_harness) [explore] {0}		
process timing		overall results
run time : 0 days, 0 hrs, 0 min, 43 sec		cycles done : 15
last new path : 0 days, 0 hrs, 0 min, 1 sec		total paths : 703
last uniq crash : none seen yet		uniq crashes : 0
last uniq hang : none seen yet		uniq hangs : 0
cycle progress	map coverage	
now processing : 261*1 (37.1%)	map density : 5.78% / 13.98%	
paths timed out : 0 (0.00%)	count coverage : 3.30 bits/tuple	
stage progress	findings in depth	
now trying : splice 14	avored paths : 114 (16.22%)	
stage execs : 31/32 (96.88%)	new edges on : 167 (23.76%)	
total execs : 2.55M	total crashes : 0 (0 unique)	
exec speed : 61.2k/sec	total tmouts : 0 (0 unique)	
fuzzing strategy yields	path geometry	
bit flips : n/a, n/a, n/a	levels : 11	
byte flips : n/a, n/a, n/a	pending : 121	
arithmetics : n/a, n/a, n/a	pend fav : 0	
known ints : n/a, n/a, n/a	own finds : 699	
dictionary : n/a, n/a, n/a	imported : n/a	
havoc/splice : 506/1.05M, 193/1.44M	stability : 99.88%	
py/custom : 0/0, 0/0		
trim : 19.25%/53.2k, n/a		
		[cpu000: 12%]

AFL++ Repository: <https://github.com/AFLplusplus/AFLplusplus>

More on fuzzing: <https://www.fuzzingbook.org/>

From Fuzzing to Security Testing

Fuzzing is just one of the various techniques to discover vulnerabilities.

Its goal is to find crashes—symptoms of a buffer overflow—or infinite loops.

However, security is not only about crashes and infinite loops... There are way more vulnerability types.

Vulnerabilities affect different functionalities or code components based on their type.

For example, classes that sanitize HTTP request input can introduce SQL Injection or XSS vulnerabilities.

Hence, classes without these responsibilities should be “immune” from such issues.

Each functionality of our system should be tested for the specific types of vulnerabilities it could be affected by.

At this point, security testing steps in! Just like traditional functional testing ensures that a particular component (a function, a method, a sequence of calls, an entire system) does what it is meant to do (i.e., adheres to its functional specification), security testing ensures that a specific component is appropriately secure.

We intend “secure” as something “hopefully free from vulnerabilities” for simplicity—recall that we cannot prove the absence of (security) bugs, and we can only prove their existence.

Security testing aims to **discover specific vulnerabilities affecting software components based on their responsibilities**, rather than going “random” as a fuzzer would.

From Fuzzing to Security Testing

Given a specific software component (e.g., function, class, use case), we model its “security aspect” as a set of properties that must hold. We can reuse all the knowledge we have about functional testing.

Functional Testing

- Test inputs are designed according to a given test selection strategy, generally guided by functional specifications or source code.
- The oracle is determined by functional specifications.
- Can target any granularity level (from unit to system).
- Generally written manually but executed automatically (e.g., JUnit).

Security Testing

- Test inputs according to a given test selection strategy, generally guided **by the “security part” of the specifications.**
- The oracle is determined **by the “security part” of the specifications.**
- Can target any granularity level (from unit to system).
- Generally written manually but executed automatically (e.g., JUnit).

Fuzzing

- Tests **“random”** inputs according to a given test generation strategy.
- The oracle is **generic, expecting not to see weird behaviors (e.g., crashes, infinite loops).**
- Can target any granularity level, **but commonly the entire system.**
- **Ready-to-use tools that may require an initial seed input.**
Willing testers could write their own custom fuzzer.

Example: Security Testing for a Login Method

Let us consider a simple Login functionality, which authenticates users using a username and a password. From a purely functional perspective, we know that:

1. The login is denied if **the username is not in the system**.
2. The login is denied if **the username is in the system**, but **the associated password does not match the supplied one**.
3. The login is approved if **the username is in the system** and **the associated password matches the supplied one**.

Assume we have written JUnit test cases for these aspects, and everything works well.

However, from a security perspective, what does this specification lack?

We have not thought about any mechanism to resist brute-force attacks!

Our login works, but it works so well that attackers can try an unlimited number of username-password combinations.

If our password requirements are not robust, attackers will eventually be able to authenticate.

A simple way to discourage these attacks is to set a limit: for example, we can block users' IP addresses after five consecutive login failures. The entry on the list lasts 20 minutes.

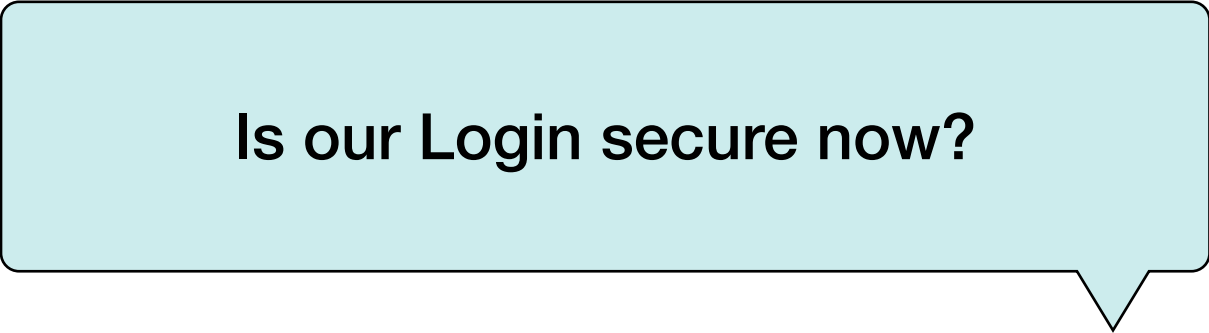
This security requirement is linked to the Login functionality!

We can quickly write test cases if we integrate it into its functional specification. Let us give an example...

Example: Security Testing for a Login Method

Let us consider a simple Login functionality, which authenticates users using a username and a password. From a purely functional perspective, we know that:

1. The login is denied if **the username is not in the system**.
2. The login is denied if **the username is in the system**, but **the associated password does not match the supplied one**.
3. The login is approved if **the username is in the system** and **the associated password matches the supplied one**.
4. The login is approved if **we have made no past attempts and provided a valid username and password**.
5. The login is approved if **we have made four previous attempts and provided a valid username and password**.
6. The login is denied after **five unsuccessful attempts within 20 minutes**.



Is our Login secure now?

Suppose the authentication system follows a role-based access control (RBAC) model.

You can log in as a “student,” while I can log in as a “teacher.”

What happens if you log in but are authenticated as a “teacher”?

This aspect should be tested as well. Nevertheless, our security specification is not complete!

Example: Security Testing for a Login Method

Is our Login secure now?

Suppose the authentication system follows a role-based access control (RBAC) model.

You can log in as a “student,” while I can log in as a “teacher.”

What happens if you log in but are authenticated as a “teacher”?

This aspect should be tested as well. Nevertheless, our security specification is not complete!

Testing even the simplest Login seems trivial, but it is not.

Fortunately, when implementing recurring functionality (e.g., Login, Forgot Password), we can follow established guidelines to define comprehensive security requirements.

For example, we could use the OWASP Cheat Sheet Series (and, by extension, all OWASP projects):

<https://cheatsheetseries.owasp.org/index.html>

Security Requirements

Security testing should not be limited to testing the security requirements associated with individual functionalities, as many other security issues can affect multiple functionalities.

A good security testing strategy also ensures that certain **undesirable conditions never happen**.

To highlight this difference, we call the security requirements bound to functionalities **positive security requirements** (tested just like traditional testing strategies).

In contrast, we call those security-threatening conditions that should never happen **negative security requirements** (more difficult to test).

For example, negative security requirements are something like “whenever a privilege of a user rises to execute sensitive functionalities, the privilege should be restored even if an exception occurs (a.k.a, Secure Error Handling)” or “all the external inputs must be sanitized.”

They seem pretty obvious and vague... and they are! So why are they important?

Because even a complete list of positive requirements and a perfect testing strategy do not guarantee security.

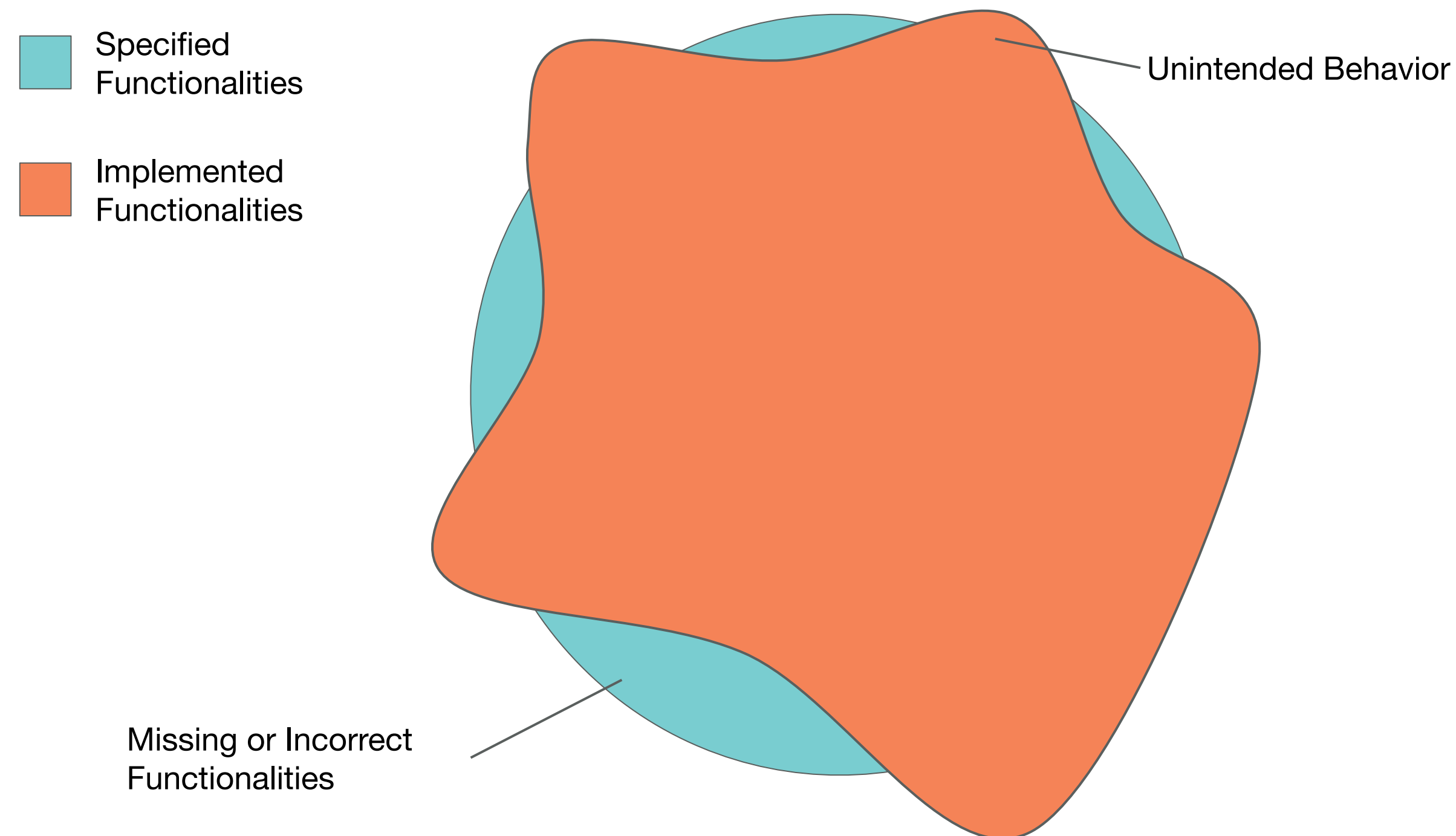
Security Requirements

A common strategy to deal with negative requirements is via the “threat and countermeasures” technique:

1. identify, even improbable, security threats,
2. define countermeasures to handle them and treat them as new functionalities,
3. test them as usual.

This “game” can be replicated multiple times, as countermeasures can be subject to other threats.

However, this aspect is more linked to Secure Requirements Engineering than Security Testing.



Testing positive security requirements reduces the “missing or incorrect functionalities” area—i.e., we can identify the lack of security features.

Testing negative security requirements reduces the “unintended behavior” area—i.e., we can identify rogue features that attackers can exploit.

Security Testing Levels

Security testing can be done at any level, from individual methods (units) to the entire system: all levels are essential for uncovering different vulnerabilities.

In particular, testing for negative requirements is typically done directly at the system level, as unintended behaviors are not always attributable to individual components.

At the integration and system levels, we can also test how well our system adheres to the defense-in-depth principles, i.e., implementing even redundant security controls at different layers.

For example, to avoid SQL Injections in web applications, we can (1) sanitize the input at the client side (through JS scripts), (2) sanitize again at the server side (replicating the same checks and adding more if possible), and (3) adopt query parameterization (e.g., using the PreparedStatement class).

Wait, aren't security tests at the system level called "penetration tests"?

There is a bit of confusion.

On the one hand, penetration testing is a structured **process** driven by **security experts**—generally **external** to the main development teams—**relying on a wide range of well-equipped tools and attack scenarios**—without necessarily writing any test code—that target the **entire application** in a **realistic** production environment, including the host platform, networks, and firewalls. The process is oriented toward negative requirements.

On the other hand, system-level security tests are written and run by the development team in a more isolated test environment, targeting the software alone and with specific expected behaviors (oracles).

From Security Testing to Vulnerability Scanning

When testing at the system level, we can observe recurring requirements, especially negative ones, i.e., common concerns across almost all applications in a specific domain.

For instance, input sanitization mechanisms — e.g., those from HTTP requests — should be implemented and tested in ALL web applications. In this respect, writing the SAME test cases multiple times would be tedious and redundant.

Fortunately, some **tools automate a specific set of system-level tests** that can be run against any application without requiring knowledge of its security requirements.

Such tools, known as *Vulnerability Scanners*, scan a target application **for vulnerabilities of various kinds** by seeding real inputs and observing responses.

Most available solutions only target web applications—i.e., they use HTTP to communicate with the application: they are commonly known as *Dynamic Application Security Testing* tools (ed: yes, I am aware of the terminology mess).

A typical vulnerability scanner

1. Selects a vulnerability type to check, e.g., XSS, Path Traversal.
2. Sends a set of predetermined inputs to its target.
3. Observes the response of the target and records everything considered interesting.

They act similarly to fuzzers, but instead of generating “random” inputs, they use more focused inputs tailored to the specific vulnerability they aim to find.

For example, if they want to find an XSS on a web page, they can send an HTTP request having `<script>alert(1);</script>` in one of the parameters.

If the HTTP response contains `<script>alert(41);</script>` in its body, it is very likely that the request was not properly sanitized, and an XSS vulnerability is present.

Vulnerability Scanning and Penetration Testing

Vulnerability scanners are among the primary weapons penetration testers use during the process.

They give an initial overview of easy-to-find vulnerabilities.

However, not all alarms represent actual vulnerabilities, so, as with static analyzers, they need human supervision alongside other tools and techniques.

A detailed list of vulnerability scanners: https://owasp.org/www-community/Vulnerability_Scanning_Tools

Shades of Vulnerability Scanning

Most scanners use predefined inputs based on proofs of concept that have demonstrated their ability to find certain vulnerability types in the past.

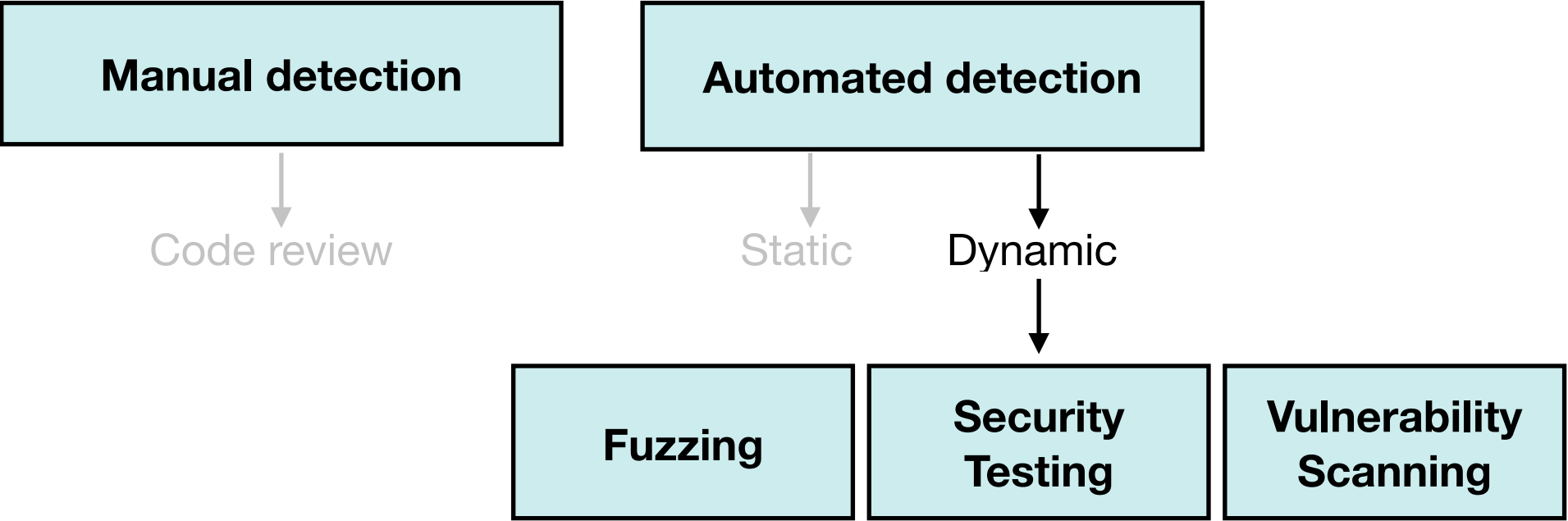
Scanners generally work in **black-box mode**: they need an interface to communicate with the target application.

However, more sophisticated tools can also rely on source code (if available) to produce better inputs, which are even more likely to uncover vulnerabilities.

For instance, a popular source code analysis technique is **taint analysis**, which analyzes how data from untrusted external inputs (the source) transforms before reaching a security-critical function (the sink).

Such a technique guides the scanner in testing with input that is more likely to reach a sink without being sanitized, hence finding additional code injection vulnerabilities.

Detecting Vulnerabilities

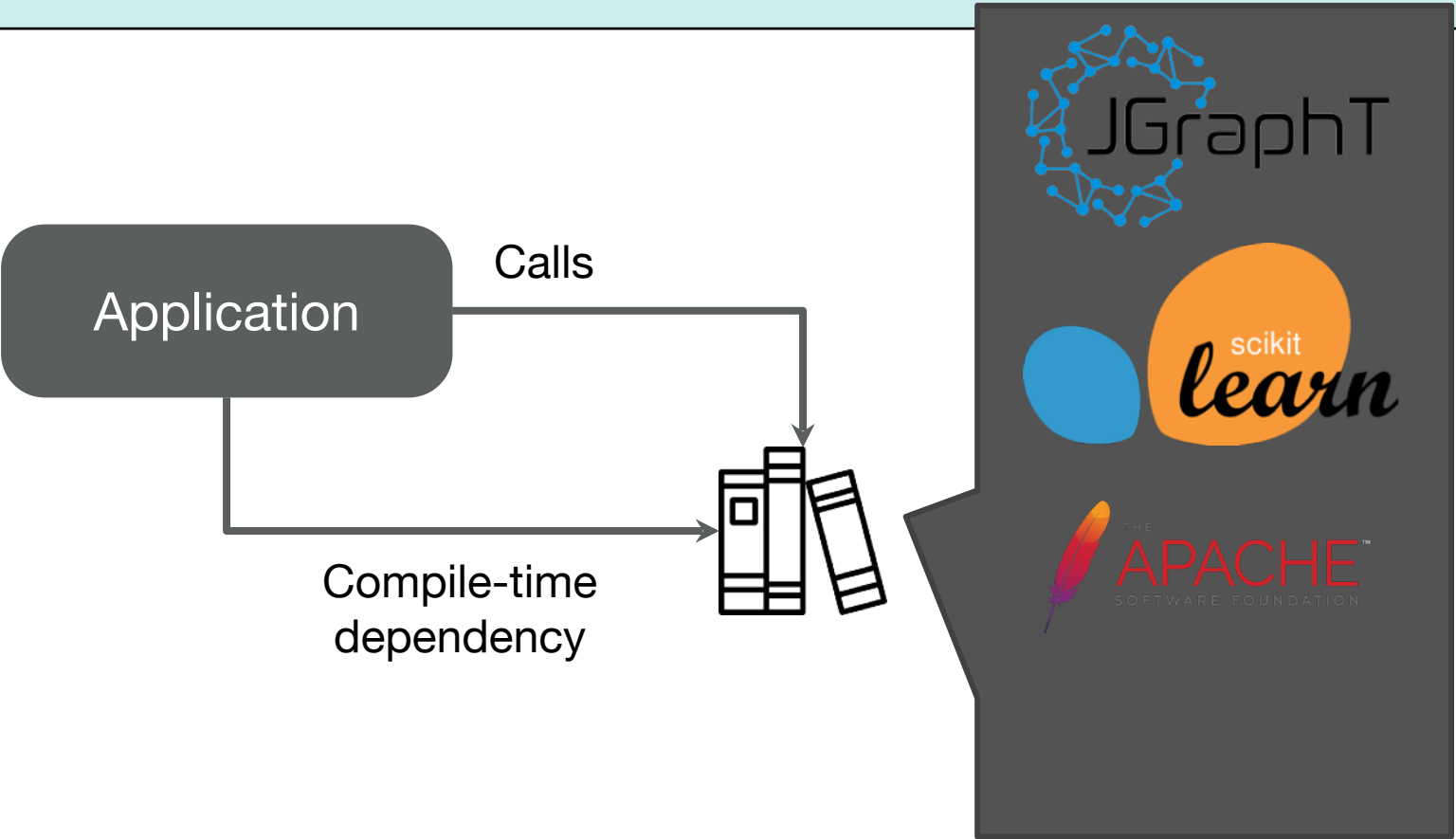


All the testing techniques we have seen focus on the source code we develop, i.e., what we have written and committed to the project’s repository.

However, software systems are primarily composed of **code we reuse** through *libraries and frameworks*.

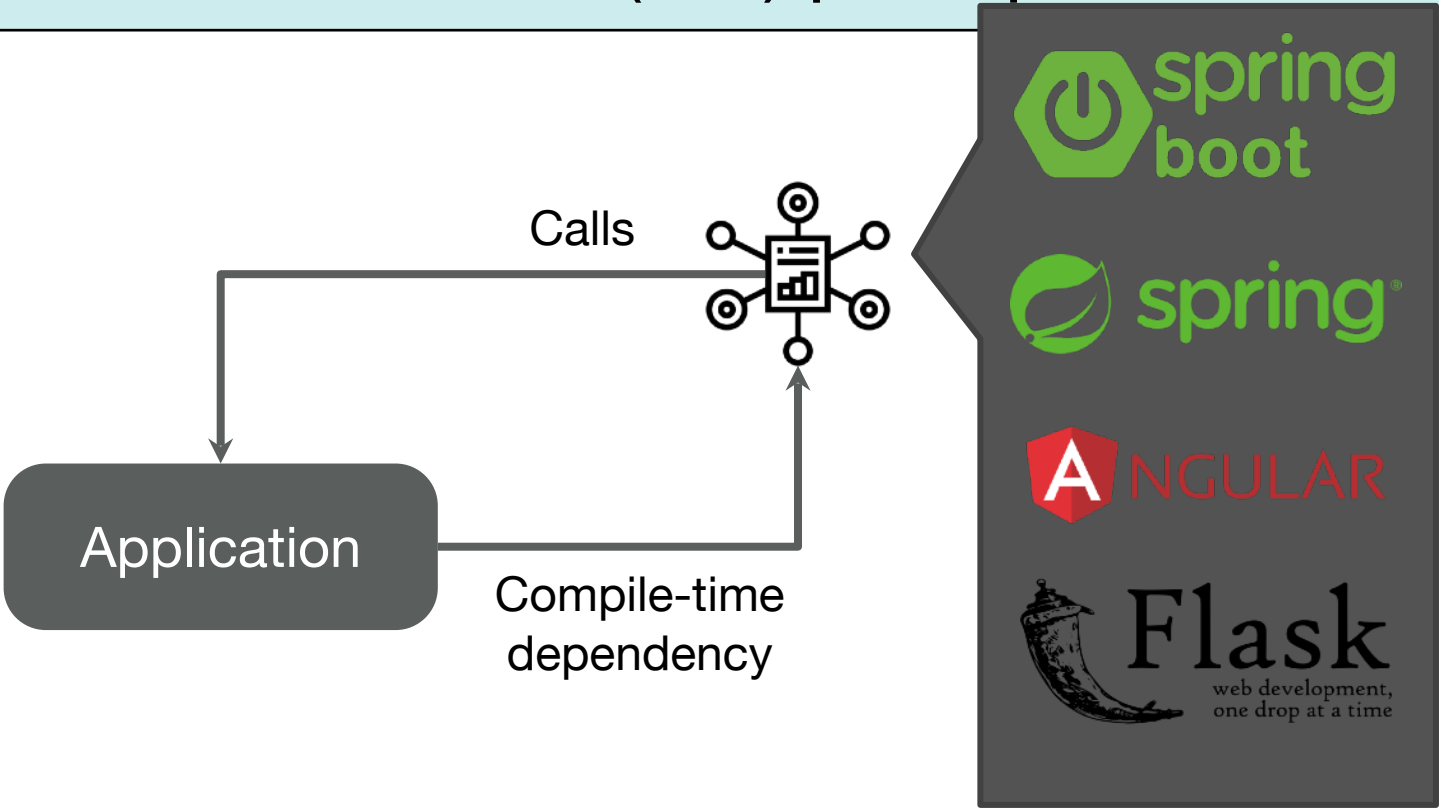
Library

A collection of reusable software components (e.g., functions, classes) generally meant for specific purposes. An application calls a library through its interface.



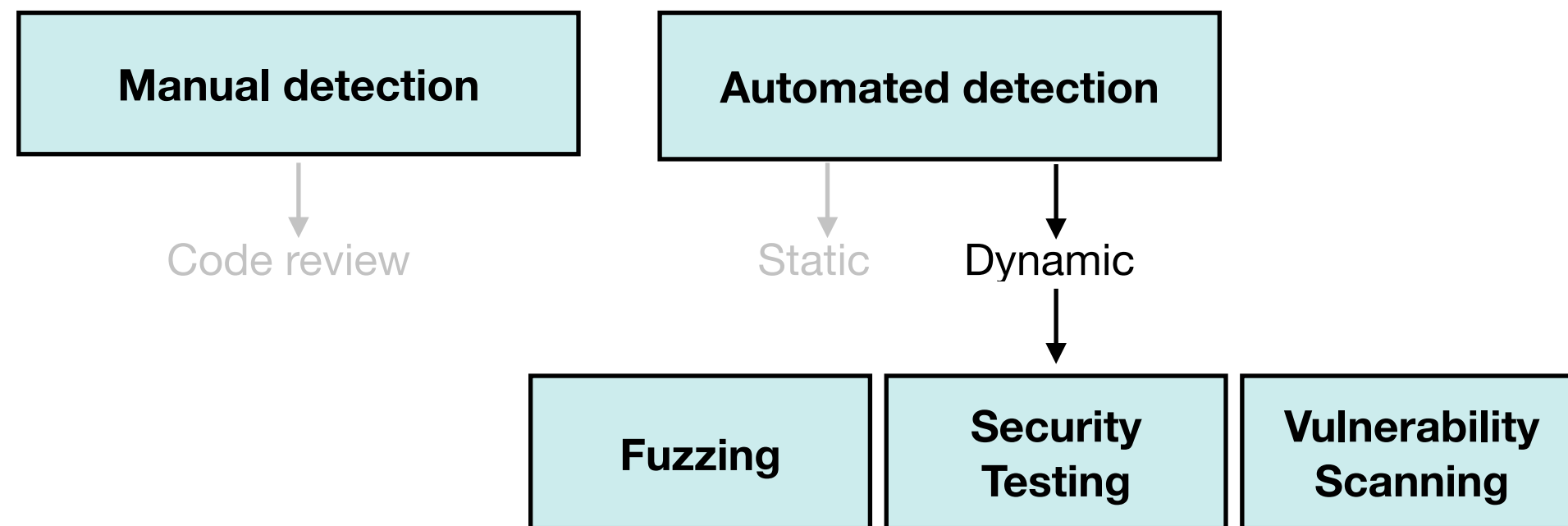
Framework

An abstract application providing generic functionalities that **can be extended—but not modified—for specific purposes**. An application relying on a framework is subject to the inversion-of-control (IoC) principle.



<https://owasp.org/www-project-top-ten/>

Detecting Vulnerabilities



Libraries and frameworks are handy for everyday tasks: we cannot think about developing a software system without relying on hundreds of them.

Libraries and frameworks are also called third-party dependencies since an application that wants to use them must include them when the entire application is built or packaged.

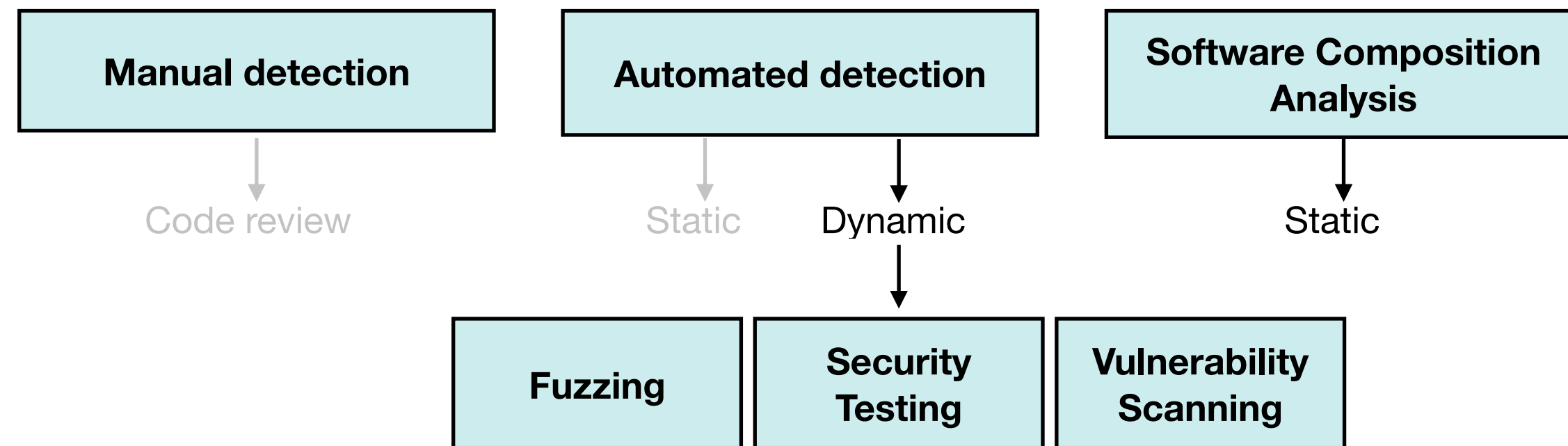
Depending on the main programming language of our application, we can add dependencies using dependency management tools, such as Maven or Gradle for Java-based systems, which help developers choose, download, and install dependencies quickly without manually managing files or environment variables.

All that glitters is not gold!

Dependencies are regularly affected by defects and vulnerabilities: we should update them as soon as new versions are available.

Nevertheless, frequent updates increase the developers' workload: they must deal with many API incompatibilities that cause build failures and new bugs. It becomes even more challenging when our dependencies form a complex dependency tree—as dependencies also have their dependencies.

Software Composition Analysis



A smart update policy should consider that **not all updates are equally important** and should rely on mechanisms to support developers in selecting urgent/critical updates, e.g., those to which we are vulnerable.

Software Composition Analysis (SCA) comes to the rescue!

Such tools can analyze the complete list of direct and transitive dependencies in our application—forming a Bill of Materials (BOM)—to identify versions affected by known vulnerabilities (i.e., those disclosed via CVE records).

The most straightforward tools report the list of vulnerable dependencies, while more complex tools provide additional support to developers in selecting which should be updated earlier than others.

In any case, an SCA tool is a nice addition to our vulnerability assessment suite.

Software Composition Analysis

Manual detection

Code review

Automated

Static analysis

Fuzzing

[Security] Bump sshpk from 1.13.1 to 1.16.1 #23

Open dependabot wants to merge 1 commit into master from dependabot/npm_and_yarn/sshpk-1.16.1

Conversation 0 Commits 1 Checks 0 Files changed 1

dependabot bot commented 3 days ago

Bumps sshpk from 1.13.1 to 1.16.1. This update includes security fixes.

Vulnerabilities fixed

Release notes

Commits

compatibility 92%

Dependabot will resolve any conflicts with this PR as long as you don't alter it yourself. You can also trigger a rebase manually by commenting @dependabot rebase .

Dependabot commands and options

[Security] Bump sshpk from 1.13.1 to 1.16.1

dependabot bot added dependencies security labels 3 days ago

All checks have passed

This branch has no conflicts with the base branch

Reviewers

Assignees

Labels

Projects

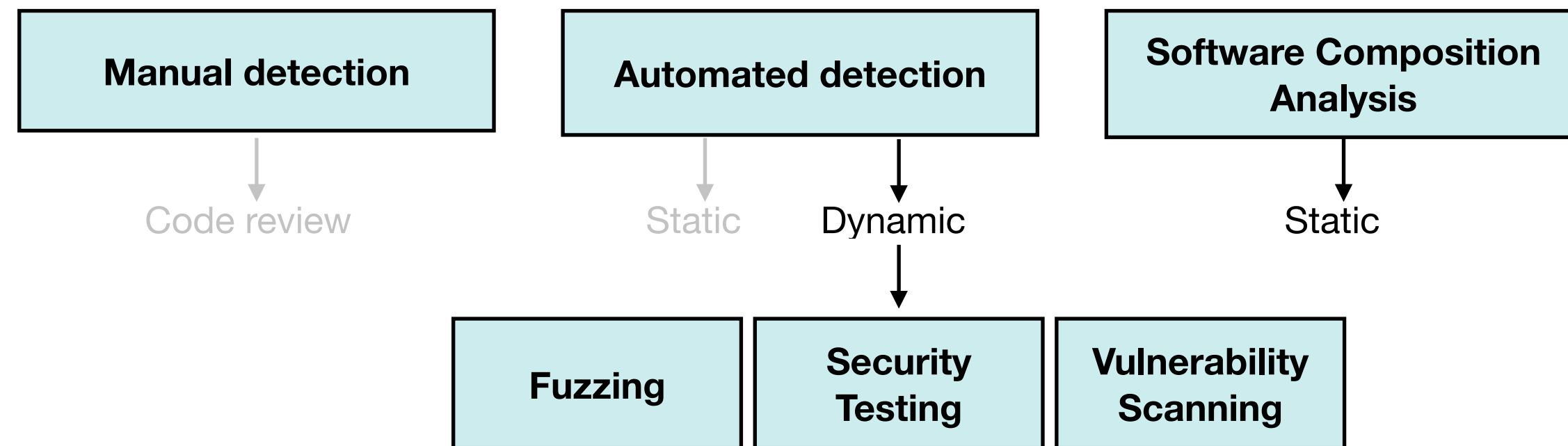
Milestone

Notifications

SCA generally performs a
projects—or analyzes the
application, the security pa
An example of a popular st
This bot proposes pull requ
A detailed list of SCA tools

.xml file for Maven-based
never used by the calling
the test suite.

Software Composition Analysis



Nevertheless, SCA tools that rely solely on static analysis are imprecise: a recent study shows that they can sometimes produce more than 70% false positives.

More sophisticated SCA tools, such as Eclipse Steady, perform fine-grained analyses to assess whether the calling Java application uses vulnerable parts of a dependency, further reducing the number of false alarms.

In short, it makes three types of analyses:

1. **Static Assessment** analyzes the static call graph to determine whether the vulnerable construct (i.e., classes) can be reached from the application classes.
2. **Dynamic Assessment** analyzes the dynamic call graph built from available test cases to determine whether vulnerable constructs are executed.
3. **Hybrid Assessment** combines static and dynamic assessment results and runs a new static assessment to further expand the search.



Beyond SCA: Software Supply Chain Security

Vulnerable dependencies seem like a trivial issue that can be easily avoided, but studies have shown that developers respond too slowly to vulnerability disclosures or are unaware of them.

Knowing this, malicious users have discovered new ways to conduct attacks.

Instead of exploiting vulnerabilities in the target application, they **interfere with the build and packaging steps to inject vulnerable code or malware**, enabling the exploitation they want.

Indeed, most third-party dependencies are distributed via public and distributed repositories—e.g., Maven Central—and their source code is generally hosted on collaboration platforms like GitHub.

Attacks take advantage of this by carrying out these attacks:

1. Deceiving developers into installing a library promising certain features, but with several exploitable vulnerabilities.
2. Creating a library with a name similar to popular ones and hoping developers mistakenly install it (*Typosquatting*).
3. Contributing to the projects of “good” libraries with “bad” commits, e.g., with vulnerabilities.

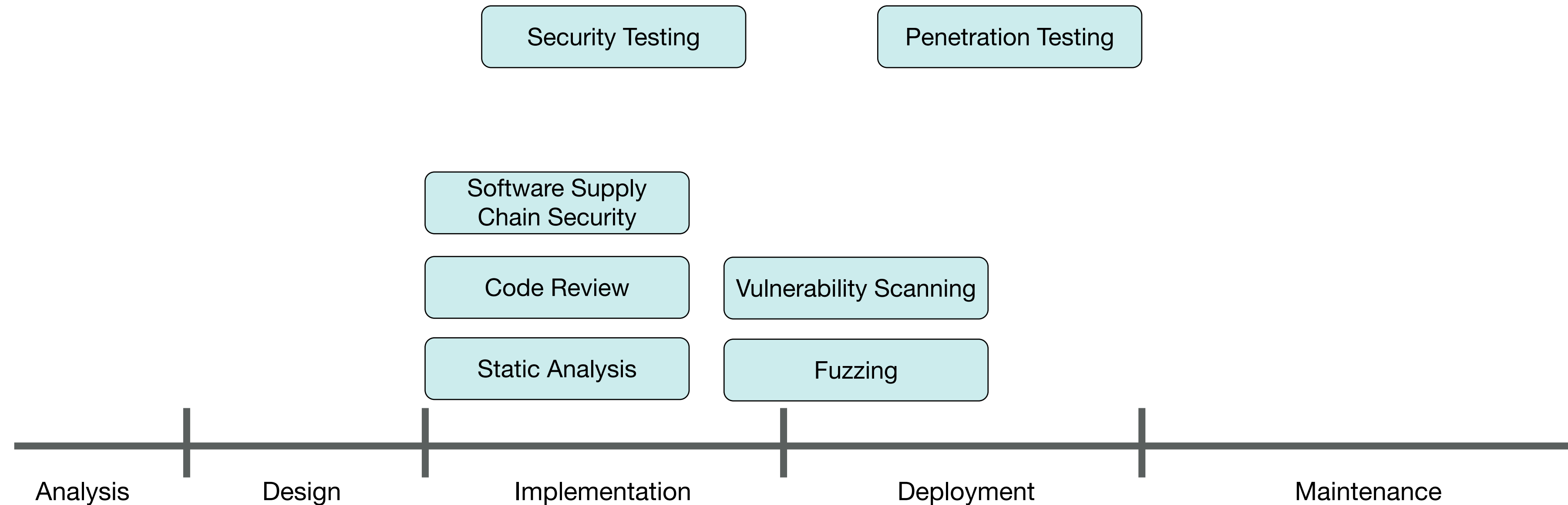
These threats and the related countermeasures pertain to Software Supply Chain Security, a new research area:

<https://slsa.dev>

<https://owasp-scvs.gitbook.io/scvs>

Joining the Pieces Together

Assuming a waterfall-like development process, we can arrange the analyses as follows.



In agile process models, automated testing techniques are included in the build and Continuous Integration pipelines.

Should we adopt them all? It depends on how dependable the system we develop is.

A highly dependable system requires robust testing and verification techniques.

To manage all these aspects, the OWASP community is the leader in defining **guidelines to support the adoption and assessment of good security practices.**

Joining the Pieces Together

All assessment techniques have their limitations:

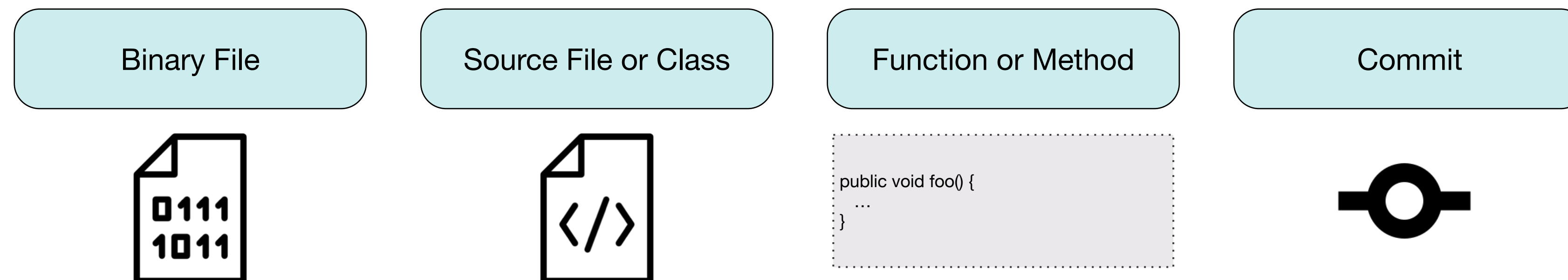
1. Code review is entirely based on human judgment and does not scale.
2. Static analysis relies on detection rules, which often result in false alarms for specific vulnerabilities.
3. Dynamic analysis might take a long time to produce relevant results.

We want alternative techniques that combine the best of all worlds:

1. Not (fully) based on human judgment.
2. Not dependent on strict detection rules.
3. Provide feedback in a reasonable time.

**Vulnerability
Prediction Models**

Through supervised machine learning algorithms, many researchers have experimented with detecting vulnerabilities at different granularity levels:





Break